

ΑΝΑΦΟΡΑ ΕΞΑΜΗΝΙΑΙΑΣ ΕΡΓΑΣΙΑΣ

Όνοματεπώνυμο	Αικατερίνη Λιάγκα	ΑΜ	031 17208		
Μάθημα	Τεχνολογία Πολυμέσων	Σχολή	Σ.Η.Μ.Μ.Υ.	Έτος	2023-2024

Η εργασία υλοποιήθηκε σε vscode χωρίς κάποιο build tool. **Προσοχή:** Για να τρέξει σε άλλο μηχανήμα πρέπει να σβηστεί το .vscode/launch.json και να δημιουργηθεί ξανά με Add Configuration. Πρέπει να προστεθεί η σειρά "vmArgs" όπως φαίνεται στο launch.json που στάλθηκε με καινούριο --module-path που οδηγεί στο τοπικό javafx-sdk.

Υλοποιήθηκαν όλες οι βασικές απαιτήσεις που περιγράφονται στην εκφώνηση. Παρακάτω περιγράφονται οι επιπλέον παραδοχές που έγιναν και εξηγείται η δομή και η λογική που ακολουθήθηκε στην υλοποίηση.

A.1. Σχεδιασμός και υλοποίηση λογικής

Η λογική υλοποιήθηκε στις κλάσεις του πακέτου src/models, ενώ δημιουργήθηκαν και custom exceptions για την διαχείριση λαθών που σχετίζονται με τις προδιαγραφές του συστήματος, τα οποία βρίσκονται στο πακέτο src/exceptions. Αποφεύχθηκε η χρήση null και επιδιώχθηκε διαχείριση με exceptions. Επίσης, όλα τα πεδία των κλάσεων είναι private και έχουμε παντού setters και getters.

- 1. Library:** Εδώ κρατώνται όλες οι λίστες Users, Admins, Books, Categories και ActiveBorrowers, με τα δεδομένα της βιβλιοθήκης και γίνεται όλη η διαχείριση των δεδομένων αυτών μέσω στατικών μεθόδων. Όλα τα πεδία και οι μέθοδοι αυτής της κλάσης είναι στατικά.
- 2. UserBase:** Είναι η κλάση γονέας του user και admin, που περιέχει όλα τα κοινά τους χαρακτηριστικά. Τα fields του είναι το boolean πεδίο isAdmin, το username (πρέπει να είναι μοναδικό και χρησιμοποιείται ως id για έναν χρήστη, πρέπει να περιέχει μόνο χαρακτήρες A-Z, a-z, 0-9, _ και να ξεκινά με γράμμα) και το password (πρέπει να περιέχει 5 ή περισσότερους χαρακτήρες). Αυτοί οι περιορισμοί ορίζονται τόσο στους setters όσο και στον constructor για να μην δημιουργούνται σε καμία περίπτωση inconsistent data και όλοι οι έλεγχοι να γίνονται συγκεντρωμένα, εσωτερικά στην κλάση. Περιέχει και τις στατικές μεθόδους αναζήτησης βιβλίων που περιγράφονται στον User, αλλά θεωρούμε ότι μπορεί να εκτελέσει και ο Admin. Στην αναζήτηση τίτλου και συγγραφέα ψάχνουμε αν το δοθέν String περιέχεται στον τίτλο ή στο όνομα συγγραφέα αντίστοιχα (έλεγχος contains όχι equals), ενώ ο έλεγχος είναι case insensitive.
- 3. User:** Προστίθενται τα χαρακτηριστικά και οι μέθοδοι για την συμπεριφορά του απλού User. Προστέθηκαν fields διεύθυνση και γενέθλια. Ελέγχουμε στον constructor και στους setters, πέραν του μοναδικού id number και email, το format του email να είναι σωστό και τα γενέθλια να μην είναι μελλοντική ημερομηνία. Ο User περιέχει και τη λίστα borrowHistory, με όλα τα ISBN των βιβλίων που έχει δανειστεί ο χρήστης, τα οποία εντάσσονται εκεί αμέσως όταν γίνεται ο δανεισμός και παραμένουν και μετά τον τερματισμό αυτού.
- 4. Admin:** Θεωρήθηκε ότι και ένας νέος admin μπορεί να κάνει sign up στο σύστημά μας, αν του έχει δοθεί ο σταθερός κωδικός της βιβλιοθήκης, που έχει τεθεί **d4yur7g** ως ένα final static field του Library. Όταν ο Admin τερματίζει έναν δανεισμό, αυτός αφαιρείται από τα ActiveBorrowers του Library. Όταν διαγράφεται ένας User διαγράφονται και όλα του τα reviews, ενώ όταν διαγράφεται ένα βιβλίο, αυτό αφαιρείται και από όλα τα borrowHistory όλων των χρηστών που το έχουν δανειστεί. Όταν αλλάζει το username ενός χρήστη ή το ISBN ενός βιβλίου αυτά πρέπει να αλλάζουν και σε όλα τα αντικείμενα που τα χρησιμοποιούν ως αναγνωριστικά του συγκεκριμένου χρήστη ή βιβλίου αντίστοιχα. Ελέγχεται επιπλέον το όνομα κάθε καινούριας κατηγορίας ή το καινούριο όνομα μιας υπάρχουσας κατηγορίας να είναι μοναδικό και όχι κενό. Για να προστεθεί ένα βιβλίο σε μια κατηγορία πρέπει αυτή να υπάρχει ήδη, ενώ δεν μπορεί να δημιουργηθεί βιβλίο χωρίς να ανήκει σε μία κατηγορία.
- 5. Book:** Αντί για χρονιά έκδοσης έχει προστεθεί ημερομηνία έκδοσης, από την οποία μπορούμε να αντλήσουμε το έτος για τις αναζητήσεις και τη σελίδα Book Details. Θεωρήθηκε ότι το ISBN πρέπει να είναι μοναδικό και χρησιμοποιείται ως αναγνωριστικό του βιβλίου. Τα αντίτυπα δεν μπορεί να είναι αρνητικά και η ημερομηνία έκδοσης δεν μπορεί να είναι μελλοντική. Όλα ελέγχονται στους setters και στον constructor. Η κλάση Book περιέχει, επίσης, μια λίστα με Review που αντιστοιχούν στο εν λόγω βιβλίο.
- 6. Review:** Subclass του Book, περιλαμβάνει το username του χρήστη που το δημιούργησε, ένα rating και ένα comment. Για να δημιουργηθεί Review πρέπει να έχει καταχωρηθεί έγκυρο rating (1-5), ενώ το

comment μπορεί να είναι και κενό. Θεωρούμε ότι για να βαθμολογήσει ο χρήστης ένα βιβλίο πρέπει να το δανείζεται τώρα (ActiveBorrow), ενώ αν προσπαθήσει να κάνει παραπάνω από ένα review για ένα βιβλίο τροποποιεί το ήδη καταχωρημένο του review. Σε κάθε περίπτωση ανανεώνεται το avg rating του βιβλίου.

7. **Category:** Κλάση με το όνομα της κατηγορίας (μια λέξη σε lowercase) και μια λίστα με ISBNs που υποδηλώνουν τα βιβλία που ανήκουν στην συγκεκριμένη κατηγορία.
8. **Borrowed:** Κλάση που αναπαριστά έναν ανοιχτό δανεισμό. Περιέχει το ISBN του βιβλίου, το username του χρήστη που έκανε το δανεισμό, την ημερομηνία δανεισμού (αρχικοποιείται αυτόματα στην ημερομηνία που δημιουργείται το αντικείμενο με το `java.time.LocalDate.now()`), καθώς και την ημερομηνία επιστροφής (είναι η ημερομηνία δανεισμού αυξημένη κατά 5 ημέρες). Στον setter της ημερομηνίας επιστροφής γίνεται έλεγχος, ώστε να μην μπορεί να τεθεί πριν την αρχική ημερομηνία επιστροφής, κάτι το οποίο δεν χρησιμοποιείται αλλά προστέθηκε για λόγους πληρότητας της λογικής.

A.2. Αποθήκευση και ανάκτηση πληροφοριών εφαρμογής

Οι συναρτήσεις serialization και deserialization υλοποιούνται στο αρχείο `DataStorageManager` του `src/models`, ενώ κάθε λίστα αποθηκεύεται στο αντίστοιχο αρχείο `.ser` του πακέτου `src/medialab`. Το deserialization καλείται, για κάθε λίστα της κλάσης `Library`, στην συνάρτησή της, `initializeData()`, που εκτελείται στην `init()` της main κλάσης `App`, ώστε να ανακτώνται τα δεδομένα με την αρχικοποίηση της εφαρμογής. Το serialization γίνεται για κάθε λίστα της `Library` στην συνάρτησή της, `saveData()`, που καλείται στην `stop()` της `App`, για την αποθήκευση δεδομένων με τον τερματισμό της εφαρμογής. Θεωρούμε ότι τα αρχεία `.ser` δίνονται άδεια στην αρχή, για αυτό δημιουργήθηκαν κάποια dummy data για ευκολότερη πλοήγηση και testing στην εφαρμογή. Αυτά δημιουργούνται στην `dummyDataCreation()` της `App`, η εκτέλεση της οποίας έγινε μόνο μία φορά και έχει τοποθετηθεί σε σχολίο για να μην εκτελεστεί ξανά. Τα δεδομένα αυτά ικανοποιούν τις παραδοχές και τους περιορισμούς της εφαρμογής.

A.3. Δημιουργία γραφικής διεπαφής

Στο πακέτο `src/views`, περιέχονται όλα τα `FXML` αρχεία (καθένα από τα οποία αντιστοιχεί σε μια σελίδα του UI) που δημιουργήθηκαν με το εργαλείο `SceneBuilder`. Για κάθε `FXML` αρχείο έχει δημιουργηθεί ο αντίστοιχος controller με το κατάλληλο όνομα, ο οποίος περιέχεται στο πακέτο `src/controllers` (π.χ. `register_user.fxml` → `RegisterUserController.java`). Σημειώνεται ότι για όλες τις λίστες έχουν δημιουργηθεί custom cells, οι controllers των οποίων ξεκινούν με `ListCell` (π.χ. `listcell_review.fxml` → `ListCellReview`).

Στα μόνα αρχεία που δεν υπάρχει αυτή η αντιστοίχιση είναι τα `admin_help.fxml` και `user_help.fxml` που είναι απόλυτα σταθερές σελίδες και στον `NavigationController.java` (NC). Ο NC είναι ένας γενικός controller που δημιουργήθηκε για την υλοποίηση στατικών συναρτήσεων που χρησιμοποιούνται συνεχώς σε όλους τους controllers, καθώς και για την αποθήκευση του `loggedUser` (ως `UserBase`) όταν κάποιος χρήστης κάνει login. Σημειώνεται ότι, όταν κάποιος χρήστης κάνει login, υπάρχουν σταθερά μενού στο UI που τα διαχειρίζονται οι `UserTemplateController` ή `AdminTemplateController`, για αυτό και κατά την αρχικοποίηση αυτών κρατείται το `borderpane` τους στον NC και αλλάζει μόνο το `center` αυτού.

Στο Sign Up υλοποιήθηκε Confirmation Password λογική, όπου ο χρήστης κατα την εγγραφή πρέπει να συμπληρώσει το ίδιο password 2 φορές, όπως γίνεται στις περισσότερες εφαρμογές. Το search που ζητείται γίνεται (για τον User και τον Admin) στη σελίδα Advanced Book Search, ενώ υπάρχει και “γρήγορη” αναζήτηση με βάση τον τίτλο, που εκτελείται με Enter, στο πάνω μενού της σελίδας. Όταν ένας Admin προσπαθεί να κάνει διαγραφή κατηγορίας, χρήστη ή βιβλίου, ζητείται πρώτα Confirmation με Alert και μόνο αν πατηθεί OK πραγματοποιείται η ενέργεια. Γενικά στους controllers γίνεται η διαχείριση των περισσότερων Exceptions, όπου δείχνονται τα κατάλληλα Alerts για να ενημερωθεί ο χρήστης για το λάθος που προκύπτει. Πολλά από τα handled Exceptions δεν θα πεταχτούν ποτέ με τον τρόπο που έχει υλοποιηθεί η εφαρμογή, αλλά τα διαχειριζόμαστε όπως και να έχει για λόγους πληρότητας στην λογική.

A.4. Λοιπές απαιτήσεις

- Έχουν ακολουθηθεί οι αρχές σχεδίασης του αντικειμενοστραφούς προγραμματισμού.
- Επιλέχθηκε η κλάση **UserBase** για τεκμηρίωση σύμφωνα με τις προδιαγραφές του javadoc.